

Name and Surname: _____

Student ITS No: _____

Qualification: _____ Year of Study: _____ Semester: _____

Assignment due date: _____ Date submitted: _____

ASSIGNMENT INSTRUCTIONS

Please tick each box to confirm completion.

Use Times New Roman font, size 12, with 1.5 line spacing throughout the document. Apply Harvard Referencing Style for all citations and references.

For essay-style assignments, please include the following sections:

- Table of Contents
- Introduction
- Main Body (with relevant subheadings)
- Conclusion
- References

Submit the assignment in PDF format on Moodle. Use the specified cover page provided.

Include a signed declaration of originality.

DECLARATION OF ORIGINALITY:

I hereby declare that this assignment is my own work and has not been copied from any other source except where due acknowledgment is made. I affirm that all sources used have been properly cited and that this submission complies with the institution’s policies on academic integrity and plagiarism.

Student Signature: _____

Date: _____

QUESTIONS ONE

[100 MARKS]

Multi-Branch Banking Management System

Background Context

The banking sector has undergone a significant digital transformation over the past two decades, with institutions moving from manual ledger systems to sophisticated computerized platforms. Modern banks require robust systems that can handle multiple account types, complex transactions, branch operations, and comprehensive reporting while maintaining the highest standards of data security and integrity.

Standard Bank, one of South Africa's leading financial institutions, has approached your software development company to create a console-based prototype of its core banking system. This prototype will be used to train new IT staff and demonstrate fundamental banking operations before they work on the main web-based system.

In this assignment, you are tasked with developing a fully functional Multi-Branch Banking Management System using C++ as a console application. This system must demonstrate your mastery of object-oriented programming, file handling, compound data structures, and control structures.

Core Requirements

(Total: 100 Marks)

1.1. Authentication & Security System

[15 MARKS]

1.1.1 Bank Teller Authentication

- Bank tellers must log in with a **Teller ID** and **Password** before accessing the system.
- Implement a Teller class with
 - Teller ID (e.g., T001, T002)
 - Full Name
 - Encrypted password (store hashed using a simple encryption algorithm)
 - Branch Code (to restrict teller access to their branch only)
- Store teller credentials in tellers.dat (binary file).
- After login, tellers can:
 - Register new customer accounts

- View customer details
- Process transactions on behalf of customers
- Generate reports for their branch

1.1.2 Customer PIN-based Authentication

- When a customer account is created, the system must generate a **5-digit unique PIN**.
- Customers must log in using
 - **Account Number** (format: ACC-BRANCH-XXXXX)
 - **5-digit PIN**
- PIN must be stored encrypted (not plain text).
- After successful login, customers can
 - View account balance
 - Perform transactions (deposit, withdrawal, transfer)
 - View account statement
 - Change PIN (with validation)
- Implement login attempt limiting (max 3 attempts before temporary lockout).

1.2. Customer Account Management

[20 MARKS]

1.2.1 Account Registration (Teller Function)

- Only authenticated tellers can register new customers.
- **Required Information**
 - Account Number (auto-generated, format: ACC-BRANCH-XXXXX)
 - Full Name (First Name and Surname)
 - South African ID Number (13 digits, validated)
 - Contact Number (10 digits)
 - Email Address (must contain @ and domain)
 - Physical Address

- Date of Birth (DD/MM/YYYY)
- Account Type (Savings, Cheque, Fixed Deposit, Student)
- Initial Deposit Amount (minimum based on account type)
- Branch Code (linked to teller's branch)
- **5-digit PIN** (auto-generated, displayed once to teller)

1.2.2 Account Types Implementation

Implement four different account types using inheritance:

- **Base Class:** Account
- **Derived Classes**
 - Savings Account
 - Cheque Account
 - Fixed Deposit Account
 - Student Account

1.3. Transaction Processing System

[25 MARKS]

1.3.1 Transaction Authentication

- All transactions require **customer PIN verification**.
- For teller-assisted transactions, the teller must verify the customer's PIN.
- For self-service (customer login), PIN is verified during the login session.

1.3.2 Transaction Types

- Cash Deposit
- Cash Withdrawal
- Fund Transfer
- Account Statement

1.3.3 Transaction Validation and Error Handling

- Use **try-catch blocks** and control structures for validation.

- Validate PIN before processing any transaction.

1.4. Multi-Branch Management

[10 MARKS]

1.4.1 Branch Operations

- Support at least 3 branches.
- Use a **struct** or **class** for Branch information.
- Store branches in an **array** or **vector**.

1.4.2 Branch Management Functions

- View All Branches
- View Branch Details
- Inter-Branch Comparison

1.5. Advanced Features

[15 MARKS]

1.5.1 Interest Calculation and Application

- Calculate interest for all eligible accounts.
- Use **loops** and **conditional statements**.

5.2 Customer Search and Filtering

- Implement comprehensive search functionality using **linear** or **binary search**.

1.6. Reporting and Analytics

[5 MARKS]

1.6.1 Management Reports

- Daily Transaction Report
- Customer Account Summary
- Branch Performance Report

1.7. File Handling Requirements

[10 MARKS]

1.7.1 Data Persistence

- Save and load data using **binary file I/O**.
- **Required Files:**

1. customers.dat – Customer accounts with encrypted PINs
 2. transactions.dat – Transaction history
 3. branches.dat – Branch information
 4. tellers.dat – Teller credentials (encrypted)
 5. system_config.txt – System settings
- Implement backup and recovery.
 - Export data to CSV and text files.

Submission Requirements

- Submit all **source code files** (.cpp, .h).
- Include **data files** (.dat, .txt) used for testing.
- Provide a **README.txt** or **userguide.pdf** explaining:
 - How to compile and run the system
 - Default teller login credentials (for testing)
 - How to create customer accounts and view generated PINs
 - How customers log in and perform transactions
- Ensure all code is **well-commented** and organized into logical modules.

N.B Please ensure that all required files are compiled and submitted in a single ZIP folder not exceeding 50 MB.

Please note that if the submitted system does not execute or fails to run, no marks will be allocated.